中图分类号：TN384　　密　　级：公　开

济南大学

本科毕业设计

基于 Unity 的类恶魔城游戏的设计与实现

学　　院　　信息科学与工程学院

专　　业　　计算机科学与技术

班　　级　　计科 2202

学　　生　　葛广一

学　　号　　202231222052

指导教师　　赵亚欧

二〇二六年五月

济南大学毕业论文（设计）诚信承诺书

本人声明所呈交的毕业论文（设计）是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得济南大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

学生签名：_____　　日期：_____年_____月_____日

济南大学毕业论文（设计）使用授权书

本人完全了解济南大学有权保留本人所送交毕业论文（设计）的复印件和电子文档，可以采用影印、缩印或其他复制手段保存论文。本人电子文档的内容和纸质

论文的内容相一致。除在保密期内的保密论文外，允许论文被查阅和借阅，可以公布（包括刊登）论文的全部或部分内容。的公布（包括刊登）授权济南大学教务处办理。

学生签名：_____　　导师签名：

日期：_____年_____月_____日　　日期：_____年_____月_____日

摘　　要

本文围绕 2D 横版动作游戏的开发需求，设计并实现了一款基于 Unity 引擎的类恶魔城动作游戏 Demo。系统采用 C#语言开发，以有限状态机组织玩家、普通敌人和 Boss 的行为逻辑，通过 Rigidbody2D、碰撞检测与动画状态控制实现移动、跳跃、冲刺、普通攻击和技能释放等核心动作。在战斗系统方面，项目实现了角色生命值、伤害判定、受击反馈、飞剑技能、黑洞技能以及 Boss 二阶段激光提示与攻击流程；在交互支持方面，完成了开始、暂停、死亡、通关等菜单界面和基础音效管理。测试结果表明，Demo 能够在 Windows 环境下独立运行，玩家可以完成基础探索、普通敌人战斗和 Boss 挑战等主要流程。该设计验证了有限状态机在 2D 动作游戏角色控制和 AI 行为管理中的可行性，也为后续扩展关卡、存档和更多技能系统提供了基础。

关键词：Unity；C#；2D 动作游戏；有限状态机；Boss 战

ABSTRACT

This thesis designs and implements a Unity-based 2D Metroidvania-style action game demo. The project is developed in C# and uses finite state machines to organize the behavior of the player, common enemies, and the boss. Core actions such as movement, jumping, dashing, wall interaction, normal attacks, and skill casting are implemented through Rigidbody2D, collision detection, and animation state control. For the combat system, the demo includes health management, damage detection, hit feedback, a flying sword skill, a blackhole skill, and a two-phase boss fight with laser warning and attack logic. For interaction support, start, pause, death, and victory menus are implemented together with basic sound effect management. The test results show that the demo can run independently on Windows and supports the main gameplay loop of exploration, enemy combat, and boss challenge. The work verifies the feasibility of applying finite state machines to 2D action game control and AI behavior management, and provides a basis for further expansion of levels, saving, and skill systems.

Key words: Unity; C#; 2D action game; finite state machine; boss fight

目　　录

摘 要.....I

ABSTRACT.....II

1 绪论.....1

1.1 选题背景与意义.....1

1.2 国内外研究现状.....2

1.3 本文主要工作.....3

1.4 论文组织结构.....3

2 需求分析.....5

2.1 系统目标.....5

2.2 功能需求分析.....5

2.3 非功能需求分析.....8

2.4 可行性分析.....8

3 系统总体设计.....10

3.1 开发环境与技术选型.....10

3.2 系统总体架构.....10

3.3 核心模块划分.....11

3.4 状态机设计.....12

3.5 资源与场景组织设计.....13

4 系统详细设计与实现.....14

4.1 玩家控制系统实现.....14

4.2 角色属性与伤害判定实现.....16

4.3 技能系统实现.....17

4.4 普通敌人AI实现.....19

4.5 Boss战斗系统实现.....20

4.6 UI与音效系统实现.....22

5 系统测试.....24

5.1 测试环境与测试方法.....24

5.2 功能测试.....24

5.3 典型问题与优化.....25

5.4 测试结果分析.....26

结　　论.....28

参 考 文 献.....29

致　　谢.....30

1 绪论

1.1 选题背景与意义

随着数字娱乐产业和独立游戏生态的发展，2D 横版动作游戏仍然保持着较强的生命力。与大型 3D 游戏相比，2D 游戏在画面资源和运行成本上更容易控制，但其对角色操作手感、碰撞反馈、敌人行为和关卡节奏的要求并不低。类恶魔城游戏通常强调横版探索、动作战斗、能力成长和 Boss 挑战，其核心体验来自玩家操作、敌人反馈和场景结构之间的配合。因此，基于 Unity 实现一款小规模但流程完整的类恶魔城动作游戏 Demo，既能够体现游戏开发的综合性，也适合作为计算机专业毕业设计的工程实践项目。

从软件工程角度看，该课题并不只是素材拼接或场景搭建，而是需要将输入响应、物理控制、动画切换、敌人 AI、技能系统、UI 交互和资源管理等多个模块组织成稳定系统。尤其在动作游戏中，角色状态切换频繁，如果缺少清晰的结构设计，代码容易出现条件判断交叉、动画状态混乱和功能扩展困难等问题。有限状态机能够将角色和敌人的行为拆分为若干独立状态，使移动、攻击、受击、死亡等逻辑分别维护，从而降低耦合度并提升可读性。

本课题以 Unity 2D 项目为载体，围绕“可运行、可演示、可说明”的毕业 Demo 目标展开。项目通过玩家控制、普通敌人、Boss 战、技能、血量 UI 和菜单流程构成基本玩法闭环，为论文中的需求分析、总体设计、详细实现和测试验证提供

了实际工程依据。通过该项目，可以训练 C#面向对象编程、Unity 组件化开发、状态机建模、动画事件调用和用户交互设计等能力，具有较好的实践意义。

1.2 国内外研究现状

在实际游戏工业中，Unity 已经成为 2D 和 3D 游戏开发中应用广泛的引擎之一。Unity 提供了 Tilemap、Animator、Rigidbody2D、Collider2D、AudioSource、Canvas 等成熟组件，能够覆盖横版动作游戏从关卡构建到角色控制、从动画播放到 UI 展示的主要需求。许多独立游戏会利用 Unity 的组件化体系快速搭建原型，再通过脚本对输入、物理参数和状态切换进行定制，以获得更贴近目标玩法的操作体验。[8-9]

在游戏 AI 设计方面，有限状态机、行为树和分层状态机是较常见的组织方式。对于毕业设计规模的 2D 动作游戏 Demo 而言，有限状态机具有实现成本低、结构清晰、便于调试的特点。普通敌人可以通过待机、巡逻、追踪、攻击、受击和死亡等状态描述行为；Boss 则可以在此基础上增加传送、施法、二阶段转场等状态，以体现更复杂的战斗节奏。与将全部逻辑写在一个 Update 函数中的方式相比，状态机能够使每个状态只关注本阶段行为，从而提升维护性。[3, 10-11]

在 2D 角色控制研究与实践中，所谓“手感”通常与输入响应速度、移动加速度、跳跃高度、碰撞检测稳定性、攻击前后摇和受击反馈有关。本文项目没有追求完整商业作品的复杂度，而是选择围绕毕业 Demo 的核心体验进行实现：保证玩家能够顺畅移动、跳跃、冲刺和攻击；敌人能够在一定范围内发现玩家并做出反应；

Boss 能够提供阶段变化和明显攻击提示。这样的取舍有利于在有限开发周期内形成稳定、可答辩的系统成果。[2,6]

1.3 本文主要工作

本文主要围绕基于 Unity 的 2D 类恶魔城动作游戏 Demo 进行设计与实现，完成的工作包括以下几个方面。

第一，完成玩家控制系统设计。项目基于 Unity 输入、Rigidbody2D、Collider2D 和 Animator 实现玩家移动、跳跃、冲刺、墙面交互、普通攻击、飞剑技能和黑洞技能，并通过玩家状态机统一管理状态切换。

第二，完成敌人与 Boss 战斗系统设计。普通敌人以骷髅敌人为代表，具备巡逻发现玩家、追踪、攻击、受击和死亡等行为；Boss 以 DeathBringer 为核心，具备近战攻击、法术攻击、传送、二阶段激光转场和死亡反馈等流程。

第三，完成战斗判定与角色属性系统设计。项目实现了 CharacterStats、PlayerStats、EnemyStats 等脚本，用于管理最大生命值、当前生命值、伤害、无敌时间和死亡状态；攻击判定通过碰撞检测、触发器和动画事件配合完成。

第四，完成 Demo 交互与发布支持。系统包含玩家血条、Boss 血条、开始菜单、暂停菜单、死亡菜单、通关菜单和基础音效管理，并完成 Windows 可执行程序打包，便于脱离 Unity 编辑器演示。

1.4 论文组织结构

本文共分为六个主要部分。第 1 章为绪论，介绍课题背景、研究现状、选题意义和主要工作。第 2 章进行需求分析，说明系统目标、功能需求、非功能需求和可行性。第 3 章进行系统总体设计，介绍开发环境、架构设计、模块划分、状态机思路和资源组织方式。第 4 章将进一步阐述玩家控制、技能、敌人 AI、Boss 战和 UI 系统的具体实现。第 5 章将给出系统测试方案和运行结果。最后在结论部分总结本设计完成的内容、存在的不足以及后续可扩展方向。

2 需求分析

2.1 系统目标

本系统的目标是实现一款能够独立运行和现场演示的 2D 类恶魔城动作游戏 Demo。Demo 不追求完整商业游戏的全部内容，而是优先保证核心玩法闭环完整，即玩家进入游戏后能够控制角色完成基础探索，与普通敌人发生战斗，并最终挑战 Boss 获得通关结果。系统需要在有限开发周期内保持功能边界清晰、运行稳定、结构便于说明，使其能够支撑毕业论文撰写和答辩演示。

根据毕业设计任务要求和当前项目实际情况，系统目标可以概括为三点：一是实现响应较好的玩家控制和战斗反馈；二是实现基于有限状态机的普通敌人与 Boss 行为逻辑；三是实现菜单、血量显示、音效和打包发布等 Demo 支撑功能。上述目标共同构成论文和作品演示的主要依据。

2.2 功能需求分析

根据游戏 Demo 的运行流程，系统功能可以划分为玩家控制、战斗技能、敌人 AI、Boss 战、UI 交互和音效发布等模块。各模块需求如表 2.1 所示。

表 2.1 系统主要功能需求

模块	功能需求	说明
玩家控制	移动、跳跃、冲刺、墙面交互、攻击	保证角色能够完成横版动作游戏的基础操作
技能系统	Dash、Sword、Blackhole	飞剑支持投掷、插入、召回和接剑，黑洞支持范围控制与持续伤害

普通敌人	巡逻、发现、追踪、攻击、受击、死亡	通过状态机组织骷髅敌人的行为流程
Boss 战	近战、施法、传送、二阶段激光、死亡	提供阶段变化和较完整的战斗演示
UI 系统	开始、暂停、死亡、通关、血条	保证游戏流程可开始、暂停、重开和结束
音效与发布	攻击、受击、死亡、菜单等音效，Windows 打包	增强反馈并支持脱离编辑器运行

2.2.1 玩家控制需求

玩家角色是系统交互的核心。角色需要能够根据键盘输入完成左右移动、跳跃、冲刺、墙面滑落、墙跳和普通攻击，并在动画状态上表现出对应动作。为了避免输入和动画互相干扰，玩家行为不宜全部堆叠在一个脚本中，而应通过状态机拆分为待机、移动、跳跃、空中、冲刺、攻击、瞄准飞剑、接剑等状态。

玩家控制还需要具备基本的容错性和反馈性。例如攻击期间应产生攻击判定，受击后应扣除生命值并触发反馈，死亡后应进入死亡表现并弹出死亡菜单。飞剑和黑洞技能需要能够由玩家输入触发，并与玩家状态切换保持一致。

2.2.2 战斗与技能需求

战斗系统需要支持玩家对敌人的普通攻击、飞剑伤害和黑洞伤害，也需要支持敌人和 Boss 对玩家造成伤害。伤害不应只表现为数值变化，还应联动受击反馈、击退方向、无敌时间、死亡判定和音效播放。对于毕业 Demo 而言，战斗系统的重点是判定清晰、反馈明确和流程稳定。

技能方面，飞剑技能需要具备瞄准轨迹、投掷、命中插入、召回和接剑等流程；

黑洞技能需要具备生成、范围显示、冻结敌人、持续造成多次伤害和结束销毁等流程。技能实现应尽量复用统一的 Skill 基类和 SkillManager 管理方式，避免不同技能之间互相耦合。

2.2.3 敌人与 Boss 需求

普通敌人需要在未发现玩家时进行基础巡逻或等待，在发现玩家后进入追踪或攻击状态。当玩家脱离检测范围时，敌人应在一定时间后退出战斗状态，避免产生全图追踪或坐标重叠抖动等不自然现象。敌人受击后需要根据攻击来源产生正确方向的击退，并在生命值归零时播放死亡表现。

Boss 战是 Demo 的主要展示内容。Boss 需要具有比普通敌人更复杂的状态切换，包括靠近玩家、近战攻击、施法攻击、传送、二阶段转场和死亡。二阶段转场需要有明显提示，例如激光瞄准线和延迟发射，使玩家能够通过冲刺进行躲避，从而提升演示效果和战斗节奏。

2.2.4 UI 与流程需求

游戏流程需要从开始菜单进入游戏，运行过程中按 Esc 能够暂停并提供继续、重新开始和退出选项。玩家死亡后应暂停游戏并显示重新开始或退出按钮；Boss 被击败后也应显示通关界面，使 Demo 具有明确的开始和结束。血条 UI 需要能够随角色生命值变化实时更新，Boss 血条应便于玩家观察战斗进程。

音效系统需要为攻击、受击、死亡、技能、Boss 激光和菜单点击等操作提供基

础反馈。虽然音效不是系统核心算法，但它能够显著增强演示时的完成度，因此作为毕业 Demo 的辅助功能进行实现。

2.3 非功能需求分析

系统的非功能需求主要包括稳定性、可维护性、可演示性和可扩展性。稳定性方面，Demo 在运行过程中不应出现阻塞流程的空引用异常、角色卡死、菜单无法响应或 Boss 无法结束等问题。可维护性方面，玩家、敌人、Boss、技能、属性和 UI 应尽量分布在独立脚本与目录中，便于后续查找和修改。

可演示性方面，项目需要能够在 Windows 环境下打包为可执行程序，使答辩或展示时不依赖 Unity 编辑器。可扩展性方面，虽然本阶段没有实现完整背包、复杂存档和大型非线性地图，但系统应保留继续添加新技能、新敌人、新关卡和存档功能的结构基础。

2.4 可行性分析

从技术可行性看，Unity 2022.3 LTS 提供了较成熟的 2D 开发能力，能够通过 Tilemap 构建关卡，通过 Rigidbody2D 和 Collider2D 处理物理与碰撞，通过 Animator 控制序列帧动画，通过 Canvas 实现菜单和血条 UI。C#语言支持面向对象封装，适合实现 Entity、CharacterStats、Skill、State 等基础类。

从时间可行性看，项目将范围控制在毕业 Demo 而非完整商业游戏，优先完成玩家控制、普通敌人、Boss 战和基础 UI 等高价值功能，降低了开发风险。从经济可

行性看，项目开发主要依赖 Unity 个人版、公开课程资源和本地开发环境，不需要额外服务器和商业部署成本。因此，本课题在毕业设计周期内具有较好的实现可行性。

3 系统总体设计

3.1 开发环境与技术选型

本项目使用 Unity 引擎和 C#语言开发，主要开发环境与工具如表 3.1 所示。[8-9, 15]

表 3.1 开发环境与技术选型

类别	工具或技术	用途
游戏引擎	Unity 2022.3 LTS	场景搭建、物理系统、动画、UI 和打包发布
开发语言	C#	编写玩家、敌人、技能、UI 和管理器脚本
物理组件	Rigidbody2D、Collider2D	处理角色移动、碰撞和触发检测
关卡工具	Tilemap	绘制横版地形、背景和碰撞区域
动画系统	Animator、Animation Clip	管理角色动作、攻击和死亡动画
版本管理	Git、GitHub	保存项目版本并支持回退与同步
发布平台	Windows	生成可直接运行的 Demo 程序

在技术选择上，Unity 能够以组件化方式组织 GameObject 及其脚本，适合毕业设计中快速搭建可运行作品。C#脚本负责描述各模块行为，例如玩家状态切换、敌人检测、Boss 攻击、技能生成、血量更新和菜单响应。相比从底层图形库开始实现，使用 Unity 可以将开发重点放在玩法逻辑和软件结构上，更符合本课题目标。[8-9]

3.2 系统总体架构

系统整体采用分层和模块化设计思想。底层由 Unity 场景、Tilemap、动画、碰撞体、音频资源和预制体构成；脚本层根据功能划分为玩家控制、敌人 AI、Boss 逻

辑、技能系统、角色属性、UI 系统和音效管理；运行层通过 Unity 生命周期函数和动画事件驱动状态更新、碰撞判定和界面刷新。

在对象组织上，玩家、普通敌人和 Boss 均继承或复用 Entity、CharacterStats 等基础能力。玩家和敌人的复杂行为由状态机管理，每个状态负责自身进入、更新和退出逻辑。技能模块通过 SkillManager 统一获取技能组件，避免玩家脚本直接管理所有技能细节。UI 模块监听角色生命值变化或游戏流程事件，并负责显示菜单和血条。

3.3 核心模块划分

根据当前项目代码结构，系统主要模块划分如表 3.2 所示。

表 3.2 系统核心模块划分

模块	代表脚本或目录	主要职责
玩家模块	Scripts/Player	玩家状态机、移动、跳跃、冲刺、攻击、技能触发
敌人模块	Scripts/Enemy/Skeleton	普通敌人的巡逻、追踪、攻击、受击和死亡
Boss 模块	Scripts/Enemy/DeathBringer	Boss 状态切换、传送、施法、二阶段激光和死亡
技能模块	Scripts/Skills	Dash、Sword、Blackhole 等技能创建、冷却和控制
属性模块	Scripts/Stats	生命值、伤害、无敌时间和死亡判定
UI 模块	Scripts/UI	血条、开始菜单、暂停菜单、死亡菜单和通关菜单
管理模块	Scripts/Managers	音效播放、玩家引用和全局辅助管理

这种模块划分使论文中的详细实现能够按照功能展开。例如玩家控制系统可以围绕 PlayerStateMachine 和各类 PlayerState 说明，敌人 AI 可以围绕 EnemyStateMachine 和 Skeleton 状态说明，Boss 战可以围绕 DeathBringer 各状态与激光控制器说明。模块之间通过明确引用和事件调用协作，减少了单个脚本承担过多职责的情况。

3.4 状态机设计

有限状态机是本系统最重要的结构之一。状态机的基本思想是将一个对象在运行过程中的行为拆分为若干互斥或阶段性状态，并在满足条件时从当前状态切换到目标状态。每个状态通常包含 Enter、Update 和 Exit 等逻辑，其中 Enter 用于初始化状态，Update 用于持续检测输入或环境条件，Exit 用于恢复或清理状态。

玩家状态机主要用于解决玩家动作频繁切换的问题。玩家可能处于待机、移动、跳跃、空中、冲刺、普通攻击、飞剑瞄准、接剑和黑洞释放等状态。如果这些逻辑全部写在一个 Update 函数中，不同输入之间容易互相影响。通过状态机拆分后，攻击状态只处理攻击期间的行为，冲刺状态只处理冲刺速度和结束条件，接剑状态只处理接剑动画和后续切换，代码含义更加清晰。

敌人和 Boss 状态机用于控制 AI 行为。普通敌人根据检测范围、玩家位置、攻击距离和战斗时间在巡逻、追踪、攻击、受击与死亡之间切换。Boss 在普通敌人基础上加入施法、传送和二阶段转场。二阶段转场通过激光预警和延迟发射制造节奏

变化，使 Boss 战不只是单一追逐和近战攻击。

3.5 资源与场景组织设计

项目资源主要放置在 Assets 目录下，脚本、动画、图像、预制体、音效和场景分别按用途组织。Scripts 目录按照 Player、Enemy、Skills、Stats、UI 和 Managers 等子目录拆分；Prefabs 目录用于保存可复用对象，例如飞剑、Boss 法术、敌人和 UI 相关预制体；Resources/Audio/SFX 目录用于存放运行时通过 AudioManager 加载的音效资源。

场景方面，Demo 以 SampleScene 作为主要演示场景，通过 Tilemap 构建地面、平台和背景层级。玩家、普通敌人、Boss、相机和 UI Canvas 在场景中协同工作。为了保证演示流程简洁，场景重点放在基础探索、普通敌人战斗和 Boss 区域，不在当前阶段扩展大型地图、复杂机关和完整存档流程。这种范围控制有助于提高作品稳定性，也使论文论述能够与实际项目保持一致。

后续章节将在总体设计基础上继续说明各模块的具体实现，包括玩家状态机、技能控制器、敌人检测逻辑、Boss 二阶段激光、血量 UI 和菜单流程等内容。

4 系统详细设计与实现

4.1 玩家控制系统实现

玩家控制系统是本项目的基础模块。Player 脚本继承自 Entity，在 Awake 方法中创建 PlayerStateMachine，并依次实例化 idleState、moveState、jumpState、airState、dashState、wallSlideState、wallJumpState、primaryAttackState、aimSwordState、catchSwordState 和 blackholeState 等状态。Start 方法中调用 stateMachine.Initialize(idleState)，使玩家默认从待机状态进入游戏。运行过程中，Player.Update 在非死亡且游戏未暂停时调用当前状态的 Update 方法，从而把不同动作逻辑交给对应状态类处理。

这种写法的核心优点是把玩家行为拆分为多个互相独立的状态。待机状态主要检测输入并切换到移动、跳跃或攻击；移动状态处理水平速度与动画参数；跳跃和空中状态处理竖直速度、落地检测和墙面检测；冲刺状态则根据 dashSpeed、dashDuration 和 dashDir 短时间改变玩家速度。不同状态之间通过 stateMachine.ChangeState 方法切换，避免在一个脚本中写大量互相嵌套的输入判断。

玩家动作还需要与动画和技能释放配合。项目通过 PlayerAnimationTriggers 接收动画事件，在普通攻击动画的关键帧执行攻击检测，在飞剑动作中调用 SkillManager.instance.sword.CreateSword 生成飞剑。玩家接剑时调用 CatchTheSword

方法，先将当前状态切换到 catchSwordState，再销毁场景中的飞剑对象并播放接剑音效。由于动画事件与状态机相互配合，系统能够保证技能释放、攻击判定和动画表现基本同步。

此外，Player 脚本中加入了 BlockActionInputFor 和 IsActionInputBlocked 方法，用于在开始菜单点击后短时间屏蔽攻击输入，避免玩家点击开始游戏时立即触发普通攻击。Die 方法负责设置死亡标记、播放死亡音效、触发死亡动画并冻结刚体，使玩家死亡后不再继续响应移动和攻击输入。整体来看，玩家控制模块围绕状态机、动画事件和刚体控制三部分展开，能够满足毕业 Demo 中移动、战斗和技能释放的需求。

（此处替换为游戏运行截图：玩家移动、跳跃、冲刺或普通攻击画面）

图 4. 1 玩家控制与基础场景运行效果

表 4.1 玩家主要状态及职责

状态名称	动画参数	主要职责
Idle / Move	Idle、Move	处理待机、水平移动和基础输入检测
Jump / Air	Jump	处理跳跃、空中下落、落地和墙面检测
Dash	Dash	根据方向和持续时间完成冲刺动作
Attack	Attack	播放普通攻击动画并通过动画事件产生伤害判定
AimSword / CatchSword	AimSword、CatchSword	控制飞剑瞄准、投掷后的接剑流程
Blackhole	Jump	释放黑洞技能并等待技能控制器结束

4.2 角色属性与伤害判定实现

角色属性系统主要由 CharacterStats、PlayerStats 和 EnemyStats 构成。CharacterStats 中定义 maxHealth、currentHealth、damage、isDead 和 isInvincible 等字段，TakeDamage 方法负责扣除生命值并通过 onHealthChanged 事件通知 UI 刷新。当 currentHealth 降低到 0 时，CharacterStats 调用 Die 方法记录死亡状态。PlayerStats 和 EnemyStats 在此基础上分别服务于玩家和敌人，使不同角色能够拥有独立血量与伤害数值。

Entity 类承担角色通用行为，包含刚体、动画、碰撞检测和受击处理等基础能力。Damage 方法提供多个重载，既可以根据 Transform 获得攻击来源，也可以直接传入 Vector2 位置和伤害值。这样设计后，普通攻击、飞剑、黑洞、Boss 法术和激光都可以复用统一的受击入口。为了使击退方向更加自然，项目根据伤害来源位置计算受击方向，而不是简单按照敌人自身朝向后退，从而避免玩家从背后攻击时敌人反向靠近玩家的问题。

伤害判定主要通过 Collider2D、Trigger 和动画事件完成。玩家普通攻击在攻击动画关键帧检测 AttackCheck 区域内的敌人，并调用 enemy.Damage(player.transform)。飞剑命中敌人或地面后停止移动，插入目标并关闭碰撞，玩家再次按键时飞剑返回；Boss 法术和二阶段激光则通过各自控制器检测玩家碰撞并调用 player.Damage。这样的实现方式使伤害来源和受击对象之间保持清晰

关系，也方便后续扩展新的攻击形式。

血量 UI 通过订阅 CharacterStats.onHealthChanged 事件刷新 Fill Image 的 fillAmount。事件驱动方式避免 UI 在每一帧主动查询角色生命值，减少了不必要的耦合。当玩家死亡或 Boss 死亡后，UI 流程脚本会根据生命值状态弹出对应菜单，使角色属性系统不仅影响战斗结果，也参与整个游戏流程控制。

表 4.2 伤害相关脚本职责

脚本	职责	说明
CharacterStats	维护生命值、伤害和死亡状态	提供 TakeDamage、IncreaseHealthBy 和 MakeInvincible 等方法
Entity	提供通用受击、击退和检测能力	玩家、普通敌人与 Boss 共享基础逻辑
PlayerAnimationTriggers	在攻击动画中触发伤害检测	保证攻击判定与动画帧同步
Sword_Skill_Controller	处理飞剑命中、插入和返回	命中敌人后调用 Enemy.Damage
DeathBringerPhaseLaser	处理二阶段激光伤害	预警结束后检测激光范围内玩家

4.3 技能系统实现

技能系统由 Skill 基类、SkillManager 以及具体技能脚本组成。SkillManager 通过 GetComponent 获取 Dash_Skill、Sword_Skill 和 Blackhole_Skill 实例，玩家状态只需要通过 SkillManager 访问技能，而不需要直接了解技能内部细节。这种结构把输入状态和技能生成逻辑分离，降低了玩家脚本的复杂度。

飞剑技能由 Sword_Skill 和 Sword_Skill_Controller 共同实现。Sword_Skill 负责读取鼠标右键输入，通过 Camera.main.ScreenToWorldPoint 将鼠标屏幕坐标转换为世界

坐标，再计算玩家到鼠标的方向作为发射方向。瞄准过程中，GenerateDots 生成轨迹点，DotsPosition 根据初速度、重力和时间计算抛物线位置，使玩家在释放前能够看到飞剑大致轨迹。释放时 CreateSword 实例化飞剑预制体，并调用 SetupSword 传入方向、重力、玩家引用和返回速度。

飞剑控制器负责飞剑发射后的状态变化。飞剑初始阶段根据 SetupSword 中的方向设置速度，命中敌人或地面后进入插入状态，停止刚体运动并关闭碰撞，避免持续触发伤害。当玩家再次按右键召回飞剑时，控制器让飞剑朝玩家位置移动；飞剑接近玩家后调用 player.CatchTheSword 切换到接剑状态并销毁飞剑对象。该流程使飞剑技能不只是一次性投射物，而是具备投掷、命中、插入、召回和接剑的完整交互。

黑洞技能由 Blackhole_Skill 和 Blackhole_Skill_Controller 实现。黑洞生成后，控制器根据 maxSize 和 growSpeed 逐渐放大自身范围，并在持续时间内按 damageInterval 对进入范围的敌人造成多次伤害。敌人进入黑洞触发器后会被加入 enemies 列表，并调用 enemy.FreezeTime(true)冻结行动；离开或黑洞结束时调用 ReleaseEnemies 恢复敌人状态。为了避免敌人在黑洞中心与玩家重叠时不断改变朝向，黑洞逻辑选择冻结敌人而不是持续吸向中心，从而保证效果稳定。

（此处替换为游戏运行截图：飞剑轨迹、飞剑命中或黑洞技能效果）

图 4.2 技能释放运行效果

表 4.3 技能实现流程

技能	触发方式	核心流程

Dash	LeftShift	检测冷却与方向，进入 Dash 状态并短时间改变速度
Sword	鼠标右键	显示轨迹点、投掷飞剑、命中插入、召回、接剑销毁
Blackhole	技能输入	生成黑洞、扩大范围、冻结敌人、造成多次伤害、缩小销毁

4.4 普通敌人 AI 实现

普通敌人以 Enemy_Skeleton 为代表。Enemy_Skeleton 在 Awake 中创建 idleState、moveState、battleState、attackState 和 stunnedState，并在 Start 中初始化为 idleState。巡逻和待机状态用于敌人未发现玩家时的基础行为；battleState 用于发现玩家后的追踪和战斗判断；attackState 用于播放攻击动画并通过动画事件检测玩家；stunnedState 用于受击后的短暂硬直。

敌人进入战斗状态的方式包括看到玩家、被玩家攻击以及受到黑洞伤害。Enemy_Skeleton 重写 Damage 和 TakeBlackholeDamage 方法，在调用 base 逻辑后执行 RefreshBattleTime，使敌人在被攻击后能够重新感知玩家并进入战斗状态。该处理解决了敌人只在主动看到玩家时才反应的问题，使玩家攻击也能触发敌人的战斗逻辑。

为避免敌人产生不自然抖动，项目在多个位置加入了容差和时间控制。当玩家与敌人横向距离非常小的时候，敌人不再频繁翻转朝向；当敌人短时间内检测不到玩家时，会经过 battleTime 后退出战斗状态，而不是立即静止或无限追踪。这样可

以减少玩家站在敌人检测边界、坐标重叠或射线断点附近时出现的左右抽搐问题。

普通敌人 AI 的实现重点不在于复杂策略，而在于行为闭环清晰。敌人能够从待机或巡逻进入追踪，靠近玩家后攻击，受击后刷新战斗时间，生命值归零后播放死亡动画并停止交互。这样的 AI 足以支撑 Demo 中的基础战斗体验，也为 Boss 战状态设计提供了结构基础。

4.5 Boss 战斗系统实现

Boss 模块以 Enemy_DeathBringer 为核心。该类继承 Enemy，并在 Awake 中创建 idleState、battleState、attackState、spellCastState、phaseTransitionState、teleportState 和 deadState。Boss 默认处于 idleState，当玩家进入 bossFightDistance 范围后，ShouldStartBossFight 返回 true，Boss 开始进入战斗流程。相比普通敌人，Boss 状态更多，切换条件也更复杂，因此更能体现有限状态机对复杂行为的组织能力。[3, 10–11]

Boss 第一阶段主要包含靠近玩家和近战攻击。为了避免 Boss 一直追到玩家身边后停住挨打，项目加入了攻击节奏控制：minAttackCooldown 和 maxAttackCooldown 用于随机生成攻击冷却，chaseBeforeAttackReadyTime 用于限制 Boss 只有在攻击即将可用时才追踪玩家。这样可以使 Boss 在攻击间隔中保持一定停顿，避免始终贴着玩家移动造成观感混乱。

Boss 二阶段由生命值阈值触发。当 stats.currentHealth 低于最大生命值的 phaseTwoHealthPercent 比例时，CheckPhaseTwo 将 Boss 切换到

phaseTransitionState。转场过程中 Boss 进入无敌状态并生成多道 DeathBringerPhaseLaser。激光控制器先显示细红色瞄准线，等待 warningTime 后逐渐变宽并进入伤害阶段。由于瞄准线在发射前短暂停止追踪玩家，玩家可以根据提示通过冲刺躲避，战斗反馈比直接瞬发伤害更清晰。

二阶段激活后，Boss 通过 ActivatePhaseTwo 提升移动速度并缩短攻击冷却，同时允许施法和传送。CastSpell 根据玩家位置和移动方向生成法术预制体，TeleportNearPlayer 则调用 FindPosition 寻找合适位置。传送位置不是简单随机赋值，而是根据玩家位置、arena 范围、地面 Raycast 结果和 OverlapBox 空间检测共同决定，从而减少传送到地底或墙体中的情况。若找到合法位置，Boss 移动到玩家附近但保持 minTeleportDistanceFromPlayer 距离，保证传送既有压迫感又不会直接重叠。

Boss 死亡时，Die 方法设置 isBossDead，播放 Boss 死亡音效，切换到 deadState，并冻结刚体、关闭碰撞体。PlayDeathAnimation 负责清理动画布尔值并播放 deathBringerExit 动画。Boss 生命值归零后，UI_BossClearMessage 在短暂延迟后显示通关菜单并暂停游戏，使 Boss 战有明确结束反馈。整体 Boss 系统通过阶段变化、攻击冷却、法术、传送和通关 UI 构成了 Demo 最完整的战斗流程。

（此处替换为游戏运行截图：Boss 近战、法术或二阶段激光提示画面）

图 4.3 Boss 战斗与二阶段攻击效果

表 4.4 Boss 主要状态及切换条件

状态	触发条件	主要行为

Idle	玩家未进入 Boss 范围	保持待机，等待战斗开始
Battle	玩家进入范围且 Boss 未死亡	面向玩家，判断追踪、攻击、施法或传送
Attack	玩家进入攻击距离且冷却结束	播放近战攻击动画并造成伤害
SpellCast	二阶段且法术冷却结束	在玩家附近生成法术攻击
Teleport	二阶段且满足传送概率和冷却	寻找玩家附近合法地面位置并传送
PhaseTransition	生命值低于阈值	进入无敌并生成多道激光提示和攻击
Dead	生命值归零	播放死亡动画，关闭碰撞，触发通关菜单

4.6 UI 与音效系统实现

UI 系统主要包括血条显示、开始菜单、暂停菜单、死亡菜单和通关菜单。UI_HealthBar 通过绑定 CharacterStats 和 Image 组件实现血量显示，当角色生命值变化时根据 currentHealth 和 maxHealth 计算 fillAmount。UI_WorldHealthBar 用于显示世界空间中的敌人血条，使普通敌人受击反馈更加直观。

菜单流程由 UI_BossClearMessage 统一管理。Awake 阶段调用 EnsureEventSystem 和 EnsureMenus，确保场景中存在 EventSystem 并创建必要菜单对象。Start 阶段根据 PlayerPrefs 中的 SkipStartMenuKey 判断是否跳过开始菜单，从而在重新开始时能够直接进入游戏而不是再次停留在主菜单。运行时按 Esc 可以打开暂停菜单，再次按 Esc 或点击继续按钮恢复游戏；玩家死亡和 Boss 被击败时分别显示失败或通关菜单。

暂停和结束流程主要通过 Time.timeScale 控制。当开始菜单或暂停菜单显示时，

游戏时间被设置为 0，使玩家、敌人和 Boss 状态更新暂停；恢复游戏时再将 Time.timeScale 恢复为 1。为了避免鼠标点击开始按钮触发玩家攻击，开始游戏时还会调用玩家的 BlockActionInputFor 短时间屏蔽动作输入。这个细节虽然较小，但能提升 Demo 演示时的自然度。

（此处替换为游戏运行截图：开始菜单、暂停菜单、死亡菜单或通关菜单界面）

图 4.4 菜单与流程控制界面

音效系统由 AudioManager 管理。AudioManager 使用单例形式存在，并通过 Resources.Load 从 Audio/SFX 目录加载音频资源。玩家攻击、玩家受击、飞剑投掷、飞剑接回、敌人死亡、Boss 法术、Boss 激光、菜单点击等操作均调用 AudioManager.PlaySFX 播放音效。音效不是核心算法，但能够增强攻击、受击和菜单操作的即时反馈，提高演示完成度。

5 系统测试

5.1 测试环境与测试方法

系统测试的目标是验证 Demo 主要流程是否能够稳定运行，重点检查玩家控制、普通敌人 AI、Boss 战、技能、UI 菜单、音效和 Windows 打包程序。测试环境为 Windows 系统，开发工具为 Unity 2022.3 LTS，测试方式以黑盒功能测试和开发过程中的回归测试为主。由于本项目为毕业设计 Demo，并未进行大规模并发测试或服务器压力测试，因此测试章节主要围绕单机运行流程展开。

测试过程分为编辑器测试和打包测试两个阶段。编辑器测试用于快速检查脚本逻辑、动画触发和碰撞判定；打包测试用于验证可执行程序在脱离 Unity 编辑器后是否能够正常启动、进入游戏、暂停、重开和退出。每次修改 Boss、技能或 UI 流程后，都需要重新运行关键路径，防止某个局部修复影响完整演示。

（此处替换为游戏运行截图：Windows 可执行程序启动或打包运行测试画面）

图 5.1 Windows 打包运行测试界面

5.2 功能测试

根据系统需求，本文选取主要功能进行测试，测试结果如表 5.1 所示。

表 5.1 系统功能测试表

测试项	测试内容	预期结果	测试结果
开始菜单	启动程序后点击开始游戏	菜单隐藏，游戏开始，玩家不立即攻击	通过
玩家移动	按方向键、跳跃键、冲刺键	角色能移动、跳跃、冲刺并播放对应动画	通过

普通攻击	玩家靠近敌人执行普通攻击	敌人扣血、受击并按正确方向后退	通过
飞剑技能	瞄准、投掷、命中、召回飞剑	轨迹点显示，飞剑可插入敌人并返回玩家	通过
黑洞技能	释放黑洞并让敌人进入范围	敌人冻结并受到多次伤害，结束后恢复	通过
普通敌人 AI	靠近、离开、攻击敌人	敌人能发现、追踪、攻击、脱战和死亡	通过
Boss 一阶段	进入 Boss 范围并战斗	Boss 启动战斗、靠近玩家并执行近战攻击	通过
Boss 二阶段	将 Boss 血量打到阈值以下	Boss 进入转场，生成激光提示和伤害判定	通过
暂停菜单	游戏中按 Esc	游戏暂停，显示继续、重开、退出按钮	通过
死亡/通关菜单	玩家死亡或 Boss 死亡	显示对应菜单并暂停游戏	通过
打包运行	运行 Windows 可执行程序	可脱离 Unity 编辑器完成主要流程	通过

5.3 典型问题与优化

在开发和测试过程中，项目出现过若干典型问题。第一类问题是空引用异常。例如飞剑预制体缺少 Sword_Skill_Controller 组件、SkillManager 未正确获取技能组件或 UI 引用未绑定时，都会导致运行时报错。针对这类问题，脚本中增加了必要的空值判断和 Debug.LogError 提示，同时在 Prefab 和场景对象上补全组件引用，使错误更容易定位。

第二类问题是敌人和玩家位置接近时产生抖动。普通敌人和 Boss 都会根据玩家相对位置翻转朝向，当二者横坐标几乎重合时，微小的位置变化会导致朝向反复切换。为此，项目加入 targetOverlapEpsilon 等容差判断，在距离过小时不再翻转朝向，

并在敌人检测不到玩家后通过 battleTime 延迟脱战，避免状态频繁进入和退出。

第三类问题是 Boss 传送位置不稳定。最初如果直接根据玩家位置设置传送坐标 Boss 可能传送到地面以下或障碍物中。优化后，Boss 通过 Raycast 向下寻找地面，并使用 OverlapBox 检查候选位置是否有足够空间，同时结合 arena 边界和 minTeleportDistanceFromPlayer 限制位置，使传送更接近玩家但不会与玩家或地形重叠。

第四类问题是技能视觉与判定不一致。飞剑如果碰撞框过大，会出现插在空气中的感觉；黑洞如果持续吸引敌人到中心，会造成敌人与玩家重叠时左右抽搐。针对飞剑，项目调小了碰撞判定并使命中后关闭碰撞；针对黑洞，项目取消持续吸向中心，改为冻结敌人并按时间造成伤害，使视觉表现和判定更稳定。

表 5.2 典型问题与处理方式

问题	原因	处理方式
空引用异常	Prefab 或场景对象缺少脚本和引用	补全组件绑定，增加空值判断和错误提示
敌人重叠抖动	横向距离过小时反复翻转朝向	加入坐标容差，距离过小时保持当前朝向
Boss 传送到地形内	传送点未检查地面和空间	通过 Raycast 找地面，OverlapBox 检查可站立空间
飞剑插空	碰撞框过大或命中后仍继续检测	调小碰撞范围，命中后停止刚体并关闭碰撞
菜单点击触发攻击	开始按钮点击和攻击输入共用鼠标事件	开始游戏后短时间屏蔽动作输入

5.4 测试结果分析

从测试结果看，系统已能够完成毕业 Demo 所需的主要流程。玩家能够从开始菜单进入游戏，完成移动、跳跃、冲刺、普通攻击和技能释放；普通敌人能够产生基础 AI 反应；Boss 能够完成一阶段、二阶段和死亡流程；UI 菜单能够覆盖开始、暂停、失败和通关场景；打包程序能够脱离 Unity 编辑器运行。

系统仍存在一定不足。首先，关卡规模较小，尚未形成完整大型非线性地图；其次，背包、完整存档和复杂能力解锁系统没有作为当前 Demo 的重点实现；再次，敌人类型和技能数量较少，战斗丰富度仍有提升空间。上述不足不会影响当前毕业 Demo 的完整演示，但可以作为后续扩展方向。

总体而言，项目在稳定性和可演示性方面达到预期目标。经过多轮修复后，玩家控制、敌人 AI、Boss 战和 UI 流程能够形成闭环，为毕业设计论文中的系统设计与实现分析提供了实际支撑。

结　论

本文以 Unity 引擎和 C#语言为基础，设计并实现了一款 2D 类恶魔城动作游戏 Demo。项目围绕毕业设计可运行、可演示和可说明的目标，完成了玩家控制、普通敌人 AI、Boss 战斗、技能系统、血量显示、菜单流程、音效管理和 Windows 打包发布等功能。系统采用有限状态机组织玩家、敌人和 Boss 行为，使复杂动作切换和 AI 逻辑能够按状态拆分，提升了代码结构清晰度和后续维护性。

在玩家控制方面，系统实现了移动、跳跃、冲刺、墙面交互、普通攻击、飞剑和黑洞技能等功能；在敌人 AI 方面，骷髅敌人能够完成巡逻、发现、追踪、攻击、受击和死亡；在 Boss 设计方面，DeathBringer 具备近战攻击、法术攻击、传送、二阶段激光转场和死亡反馈；在 UI 方面，系统实现了开始、暂停、死亡和通关菜单，并通过血条显示角色生命值。测试结果表明，Demo 能够在 Windows 环境下独立运行并完成主要战斗流程。

由于开发周期有限，本项目仍有不足之处。例如关卡规模较小，未实现完整大型地图和复杂能力门锁机制；存档和背包系统仅作为后续扩展方向，没有在当前 Demo 中完整展开；敌人种类和技能种类还不够丰富，战斗平衡也有进一步打磨空间。后续可以在现有架构基础上继续增加地图区域、能力解锁、道具收集、更多敌人类型和正式存档系统，使项目从毕业 Demo 进一步扩展为更完整的 2D 动作游戏。

参 考 文 献

[1] Bhosale T, Kulkarni S, Patankar S N. 2D Platformer Game in Unity Engine[J]. International Research Journal of Engineering and Technology, 2018, 5(4): 3021-3024.

[2] Pichlmair M, Fasterholdt M, Holmgård C. You say jump, I say how high? Operationalising the game feel of jumping[C]//Proceedings of DiGRA/FDG. 2016.

[3] Millington I. Artificial Intelligence for Games[M]. 3rd ed. Boca Raton: CRC Press, 2019.

[4] Gregory J. 游戏引擎架构[M]. 叶劲峰, 等译. 北京: 电子工业出版社, 2014.

[5] Nystrom R. 游戏编程模式[M]. 侯金标, 译. 北京: 人民邮电出版社, 2015.

[6] Swink S. Game Feel: A Game Designer's Guide to Virtual Sensation[M]. Burlington: Morgan Kaufmann, 2008.

[7] Schell J. The Art of Game Design: A Book of Lenses[M]. 3rd ed. Boca Raton: CRC Press, 2019.

[8] Unity Technologies. Unity User Manual 2022.3 LTS[EB/OL]. [2026-05-13]. https://docs.unity3d.com/2022.3/Documentation/Manual/index.html.

[9] Unity Technologies. Unity Scripting API 2022.3 LTS[EB/OL]. [2026-05-13]. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/.

[10] Bourg D M, Seemann G. AI for Game Developers[M]. Sebastopol: O'Reilly Media, 2004.

[11] Buckland M. Programming Game AI by Example[M]. Plano: Wordware Publishing, 2004.

[12] Rabin S. Introduction to Game Development[M]. 2nd ed. Boston: Cengage Learning, 2010.

[13] Adams E. Fundamentals of Game Design[M]. 3rd ed. Berkeley: New Riders, 2014.

[14] Trefry G. Casual Game Design: Designing Play for the Gamer in All of Us[M]. Burlington: Morgan Kaufmann, 2010.

[15] Microsoft. C# documentation[EB/OL]. [2026-05-13]. https://learn.microsoft.com/dotnet/csharp/.

致　　谢

本次毕业设计从选题、功能实现到论文撰写，得到了指导教师的耐心指导和帮助。在项目开发过程中，老师对课题方向、系统功能边界和论文结构提出了重要建议，使我能够将 Unity 项目开发与毕业论文写作结合起来，逐步完成从 Demo 实现到文档整理的工作。

同时，感谢在学习和开发过程中提供帮助的同学和朋友。在调试玩家控制、敌人 AI、Boss 战和 UI 菜单时，他们的试玩反馈帮助我发现了许多细节问题，也促使我不断完善 Demo 的稳定性和演示效果。

最后，感谢大学期间所有授课教师的培养。通过本次毕业设计，我对面向对象程序设计、Unity 组件化开发、有限状态机、游戏交互和项目版本管理有了更完整的认识。这些经历将成为今后继续学习和实践的重要基础。